Linux下*Makefile*配置
Windows下Makefile配置
Random,随机数生成模板
acm代码模板
快读快写
排序与高精度
    归并排序求逆序对
    高精度加法
    高精度减法
    高精乘高精
异或哈希
普通莫队
单点修改莫队
CDQ分治(三维偏序)

算法板子

Linux下*Makefile*配置

```
# 最简版(需要acm.cpp,in,out,Answer)
CXX := g++
CXXFLAGS := -Wall -std=c++17 -lm -O3 -Ofast -fsanitize=address -fsanitize=undefined -fno-omit-frame-pointer

.DEFAULT_GOAL := run

build/acm: acm.cpp Makefile
    @mkdir -p build
    $(CXX) $< -o $@ $(CXXFLAGS)

# 普通运行
.PHONY: run
run: build/acm
    @build/acm < in
```

```
CXX := g++
CXXFLAGS := -Wall -std=c++17 -lm -O3 -Ofast
TEST ?= 100

.DEFAULT_GOAL := run

build/acm: acm.cpp
    @mkdir -p build
    $(CXX) $< -o $@ $(CXXFLAGS)

build/std: std.cpp
    @mkdir -p build
    $(CXX) $< -o $@ $(CXXFLAGS)

```

```
build/random: random.cpp
    @mkdir -p build
    $(CXX) $< -o $@ $(CXXFLAGS)

.PHONY: run
run: build/acm
    @./build/acm < in

.PHONY: dp
dp: build/acm build/std build/random
    @for i in $$(seq 1 $(TEST)); do \
        ./build/random > in; \
        ./build/acm < in > out; \
        ./build/std < in > stdout; \
        if ! diff -w out stdout > /dev/null; then \
            echo "Wrong Answer on test $$i"; \
            echo "Input:"; cat in; \
            echo "ACM output:"; cat out; \
            echo "STD output:"; cat stdout; \
            exit 1; \
        fi; \
    done
    @echo "Accepted, passed $(TEST) tests"
```

```

# 完整版(还需要std.cpp,random.cpp,stdout)
CXX := g++
CXXFLAGS := -Wall -std=c++17 -lm -O3 -Ofast -fsanitize=address -fsanitize=undefined -fno-omit-frame-pointer

.DEFAULT_GOAL := run

build/acm: acm.cpp Makefile
    @mkdir -p build
    $(CXX) $< -o $@ $(CXXFLAGS)

build/std: std.cpp Makefile
    @mkdir -p build
    $(CXX) $< -o $@ $(CXXFLAGS)

build/random: random.cpp Makefile
    @mkdir -p build
    $(CXX) $< -o $@ $(CXXFLAGS)

# 普通运行
.PHONY: run
run: build/acm
    @build/acm < in

# 测试样例
.PHONY: check
```

```
check: build/acm
    @build/acm < in > out
    @if ! diff -q out Answer > /dev/null; then \
        echo "Wrong Answer"; \
        echo "Input:"; \
        cat in; \
        echo "Output:"; \
        cat out; \
        echo "Answer:"; \
        cat Answer; \
        exit 1; \
    fi
    @echo "Accepted"

# 测试特殊点，与暴力std比较
.PHONY: cp
cp: build/acm build/std
    @build/acm < in > out
    @build/std < in > stdout
    @if ! diff -q out stdout > /dev/null; then \
        echo "Wrong Answer"; \
        echo "Input:"; \
        cat in; \
        echo "ACM output:"; \
        cat out; \
        echo "STD output:"; \
        cat stdout; \
        exit 1; \
    fi
    @echo "Accepted"

# 对拍
.PHONY: dp
dp: build/acm build/std build/random
    @echo "Running test cases..."
    @for i in $$(seq 1 10); do \
        echo "[Test $$i]"; \
        build/random > in; \
        build/acm < in > out; \
        build/std < in > stdout; \
        if ! diff -q out stdout > /dev/null; then \
            echo "Wrong Answer on test $$i"; \
            echo "Input:"; \
            cat in; \
            echo "ACM output:"; \
            cat out; \
            echo "STD output:"; \
            cat stdout; \
            exit 1; \
        fi; \
    done
    @echo "Accepted"
```

Windows下Makefile配置

```
CXX := g++
CXXFLAGS := -Wall -std=c++23 -lm -O3
TARGET_DIR := build

.DEFAULT_GOAL := run

ensure-dir:
    @mkdir -p "$(TARGET_DIR)"

$(TARGET_DIR)/acm.exe: acm.cpp Makefile | ensure-dir
    $(CXX) $< -o $@ $(CXXFLAGS)
# 普通运行
.PHONY: run
run: $(TARGET_DIR)/acm.exe
    @./$(TARGET_DIR)/acm.exe < in
```

Random,随机数生成模板

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

i64 rand_int(i64 l, i64 r) {
    return uniform_int_distribution<i64>(l, r)(rng);
}

void solve() {

}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    solve();
    return 0;
}
```

acm代码模板

```
#include<bits/stdc++.h>
#define IOS ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
#define int long long
#define inf 0x3f3f3f3f
```

```cpp
#define INF 4557430888798830399
using ull = unsigned long long;
using pii = std::pair<int,int>;
typedef complex<double> cp;
const int N = 1e6 + 7;
const int M = 2e6 + 7;
const int mod = 998244353;
const double pi = acos(-1);
const double e = exp(1);

void solve()
{

}
signed main()
{
    // init();
    IOS;
    int T = 1;cin >> T;
    while(T--)
    {
        solve();
    }
    return 0;
}
```

快读快写

```cpp
inline int read()
{
    int x = 0,f = 1;
    char ch = getchar();
    while(ch < '0' || ch > '9')
    {
        if(ch == '-')
            f = -1;
        ch = getchar();
    }
    while(ch >= '0' && ch <= '9')
        x = x * 10 + ch - '0',ch = getchar();
    return x * f;
}
void print(int x)
{
    if(x < 0)
        putchar('-'),x = -x;
    if(x > 9)
        print(x / 10);
    putchar(x % 10 + '0');
    return;
}
```

排序与高精度

归并排序求逆序对

```
int a[N],c[N];
int cnt;
void msort(int l,int r)
{
    if(l >= r)return;
    int i = l,mid = (l + r) >> 1,j = mid + 1,k = l;
    msort(l,mid),msort(mid + 1,r);
    while(i <= mid && j <= r)
    {
        if(a[i] <= a[j])c[k++] = a[i++];
        else c[k++] = a[j++],cnt += mid - i + 1;
    }
    while(i <= mid)c[k++] = a[i++];
    while(j <= r)c[k++] = a[j++];
    for(int x = l;x <= r;x++)a[x] = c[x];
}
```

高精度加法

```
map<char, int>mp;
void init()
{
    for (int i = 0; i <= 9; i++)mp['0' + i] = i;
    for (int i = 0; i <= 25; i++)
    {
        mp['A' + i] = i + 10;
        mp['a' + i] = i + 36;
    }
}
string add(string a, string b, int z)//z是进制
{
    string ans = "";
    int c = 0;
    for (int i = a.size() - 1, j = b.size() - 1; i >= 0 || j >= 0; i--, j--)
    {
        if (i >= 0 && j >= 0)
        {
            int a1 = mp[a[i]], b1 = mp[b[j]];//映射的是字母代表的数
            int sum = a1 + b1 + c;
            if (sum - z >= 0)
            {
                c = 1;
            }
            else c = 0;
            sum %= z;
            if (sum <= 9)ans = (char)('0' + sum) + ans;
            else if (sum <= 35)ans = (char)('A' + sum - 10) + ans;
```

```
            else ans = (char)('a' + sum - 36) + ans;
        }
        else if (j >= 0)
        {
            int a1 = 0, b1 = mp[b[j]];//映射的是字母代表的数
            int sum = a1 + b1 + c;
            if (sum - z >= 0)
            {
                c = 1;
            }
            else c = 0;
            sum %= z;
            if (sum <= 9)ans = (char)('0' + sum) + ans;
            else if (sum <= 35)ans = (char)('A' + sum - 10) + ans;
            else ans = (char)('a' + sum - 36) + ans;
        }
        else
        {
            int a1 = mp[a[i]], b1 = 0;
            int sum = a1 + b1 + c;
            if (sum - z >= 0)
            {
                c = 1;
            }
            else c = 0;
            sum %= z;
            if (sum <= 9)ans = (char)('0' + sum) + ans;
            else if (sum <= 35)ans = (char)('A' + sum - 10) + ans;
            else ans = (char)('a' + sum - 36) + ans;
        }
    }
    if (c)
        ans = '1' + ans;
    return ans;
}
```

高精度减法

```
bool cmp(vector<int>& A, vector<int>& B)
{
    if (A.size() != B.size()) return A.size() > B.size();
    for (int i = A.size() - 1; i >= 0; --i)
        if (A[i] != B[i])
            return A[i] > B[i];

    return true;
}
vector<int> sub(vector<int>& A, vector<int>& B)
{
    vector<int> C;
    int t = 0;
    for (int i = 0; i < A.size(); ++i)
```

```
    {
        t = A[i] - t;
        if (i < B.size()) t -= B[i];
        C.push_back((t + 10) % 10);
        if (t >= 0) t = 0;
        else t = 1;
    }
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}
void jiefa(string a,string b)
{
    vector<int> A, B;
    for (int i = a.size() - 1; i >= 0; --i) A.push_back(a[i] - '0');
    for (int i = b.size() - 1; i >= 0; --i) B.push_back(b[i] - '0');
    vector<int> C;
    if (cmp(A, B)) C = sub(A, B);
    else cout << "-", C = sub(B, A);
    for (int i = C.size() - 1; i >= 0; --i) cout << C[i];
    cout << endl;
}
int main()
{
    string a,b;
    cin >> a >> b;
    jiefa(a,b);
    return 0;
}
```

高精乘高精

```
map<char, int>mp;
void init()
{
    for (int i = 0; i <= 9; i++)mp['0' + i] = i;
    for (int i = 0; i <= 25; i++)
    {
        mp['A' + i] = i + 10;
        mp['a' + i] = i + 36;
    }
}
string cheng(string a_s,string b_s,int z)//z是进制
{
    string ss = "";
    int a[521] = {0},b[521] = {0},ans[521 * 2] = {0};
    int len_a,len_b,len,carry = 0,i,j;
    len_a = a_s.length();
    len_b = b_s.length();

    for(i = 0;i < len_a;i++) a[i] = mp[a_s[len_a - 1 - i]];//映射的是字母代表的数
    for(i = 0;i < len_b;i++) b[i] = mp[b_s[len_b - 1 - i]];//映射的是字母代表的数
    for(i = 0;i < len_b;i++){
```

```
        for(j = 0;j < len_a || carry != 0;j++){
            ans[i + j] += a[j] * b[i] + carry;
            carry = ans[i + j] / z;
            ans[i + j] %= z;
        }
    }
    len = i + j;
    while(ans[len] == 0 && len > 0) len--;
    for(i = len;i >= 0;i--){
        if(ans[i] <= 9)ss += (char)('0' + ans[i]);
        else if(ans[i] <= 35)ss += (char)('A' + ans[i] - 10);
        else ss += (char)('a' + ans[i] - 36);
    }
    return ss;
}
```

异或哈希

```
用哈希降低异或的冲突，通常用来解决一个数出现的次数为奇数还是偶数的情况
const int N = 1e6 + 7;
mt19937_64 rnd(time(0));//
int p[N];//
signed main()
{
    IOS;
    for(int i = 1;i < N;i++)p[i] = rnd();//
    int T = 1;cin >> T;
    while(T--)
    {
        solve();
    }
    return 0;
}

ull p[N];
void init()
{
    p[0] = 1;
    for(int i = 1;i < N;i++)
        p[i] = p[i - 1] * 998244353 + i;
}
```

普通莫队

```
int base;//分块(n / sqrt(m))
struct query
{
    int l;
    int r;
    int id;
    bool operator<(const query &tp)const{
```

```
        if(l / base != tp.l / base)return l < tp.l;
        return (l / base) & 1 ? r < tp.r : r > tp.r;
    };
};
void solve()
{
    int n,m;
    cin >> n >> m;
    base = max(1LL,(int)(n / sqrt(m)));//注意0
    vector<int>a(n + 1);
    vector<query>que(m + 1);
    vector<int>ans1(m + 1),ans2(m + 1),col(n + 1);
    for(int i = 1;i <= n;i++)cin >> a[i];
    for(int i = 1;i <= m;i++)
    {
        int l,r;cin >> l >> r;
        que[i] = {l,r,i};
    }
    sort(que.begin() + 1,que.end());
    int sum = 0;
    //更改
    auto move = [&](int x,int sign) -> void
    {
        if(sign == -1)col[x]--;
        sum += col[x] * sign;
        if(sign == 1)col[x]++;
    };
    for(int i = 1,l = 1,r = 0;i <= m;i++)
    {
        if(que[i].l == que[i].r)
        {
            ans1[que[i].id] = 0;
            ans2[que[i].id] = 1;
            continue;
        }

        while(l > que[i].l)move(a[--l],1);
        while(r < que[i].r)move(a[++r],1);
        while(l < que[i].l)move(a[l++],-1);
        while(r > que[i].r)move(a[r--],-1);
        ans1[que[i].id] = sum;
        ans2[que[i].id] = (r - l + 1) * (r - l) / 2;
    }
    for(int i = 1;i <= m;i++)
    {
        if(!ans1[i])ans2[i] = 1;
        else
        {
            int g = gcd(ans1[i],ans2[i]);
            ans1[i] /= g;
            ans2[i] /= g;
        }
        cout << ans1[i] << "/" << ans2[i] << endl;
```

```
    }
}
```

单点修改莫队

```
int base;//分块
struct query
{
    int l;
    int r;
    int t;
    int id;
    bool operator<(const query& tp)const{
        if(l / base != tp.l / base)return l < tp.l;
        if(r / base != tp.r / base)return r < tp.r;
        return t < tp.t;
    };
};
int cnt[N];
void solve()
{
    int n,m;
    cin >> n >> m;
    base = max(1LL,(int)pow(n,2.00 / 3.00));
    vector<int>a(n + 1);
    for(int i = 1;i <= n;i++)cin >> a[i];
    vector<PA>upt(1);int times = 0;//修改
    vector<query>que(1);int tot = 0;//询问
    for(int i = 1;i <= m;i++)
    {
        char c;cin >> c;
        if(c == 'Q')//询问
        {
            int l,r;
            cin >> l >> r;
            que.push_back({l,r,times,++tot});
        }
        else//修改
        {
            int p,x;
            cin >> p >> x;
            upt.push_back({p,x});
            times++;
        }
    }
    //Mo
    sort(que.begin() + 1,que.end());
    vector<int>ans(tot + 1);
    int sum = 0;
    auto move = [&](int x,int sign)
    {
        if(sign == -1)cnt[x]--;
```

```
        sum += (!cnt[x]) * sign;
        if(sign == 1)cnt[x]++;
    };
    for(int i = 1,l = 1,r = 0,now = 0;i <= tot;i++)
    {
        while(l > que[i].l)move(a[--l],1);
        while(r < que[i].r)move(a[++r],1);
        while(l < que[i].l)move(a[l++],-1);
        while(r > que[i].r)move(a[r--],-1);
        while(now < que[i].t)
        {
            now++;
            if(l <= upt[now].first && upt[now].first <= r)
            {
                move(upt[now].second,1);
                move(a[upt[now].first],-1);
            }
            swap(upt[now].second,a[upt[now].first]);
        }
        while(now > que[i].t)
        {
            if(l <= upt[now].first && upt[now].first <= r)
            {
                move(upt[now].second,1);
                move(a[upt[now].first],-1);
            }
            swap(upt[now].second,a[upt[now].first]);
            now--;
        }
        ans[que[i].id] = sum;
    }
    for(int i = 1;i <= tot;i++)
        cout << ans[i] << endl;
}
```

CDQ分治(三维偏序)

```
struct node
{
    int a;
    int b;
    int c;
    int res;
    int cnt;
    bool operator!=(const node &tp)const{
        if(tp.a != a || tp.b != b || tp.c != c)
            return true;
        return false;
    }
};
bool cmpa(node x,node y)
{
```

```
    if(x.a != y.a)return x.a < y.a;
    if(x.b != y.b)return x.b < y.b;
    return x.c < y.c;
}
bool cmpb(node x,node y)
{
    if(x.b != y.b)return x.b < y.b;
    return x.c < y.c;
}
struct finwick
{
    vector<int>t;
    int n;
    finwick(int len)
    {
        init(len);
    }
    void init(int len)
    {
        n = len;
        t.resize(n + 1);
    }
    int lowbit(int x)
    {
        return x & (-x);
    }
    void add(int x,int k)
    {
        for(int i = x;i <= n;i += lowbit(i))
            t[i] += k;
    }
    int query(int l,int r)
    {
        int ans = 0;
        for(int i = r;i >= 1;i -= lowbit(i))
            ans += t[i];
        for(int i = l - 1;i >= 1;i -= lowbit(i))
            ans -= t[i];
        return ans;
    }
};
void solve()
{
    int n,k;
    cin >> n >> k;
    vector<node>tp(n + 2),arr(1);
    for(int i = 1;i <= n;i++)
    {
        int a,b,c;
        cin >> a >> b >> c;
        tp[i] = {a,b,c,0,0};
    }
    sort(tp.begin() + 1,tp.begin() + 1 + n ,cmpa);
```

```
    int t = 0;
    for(int i = 1;i <= n;i++)
    {
        t++;
        if(tp[i] != tp[i + 1])
        {
            node tmp = tp[i];
            tmp.cnt = t;
            arr.push_back(tmp);
            t = 0;
        }
    }
    int len = arr.size() - 1;
    finwick fi = finwick(2e5);
    auto cdq = [&](auto &&self,int l,int r) -> void
    {
        if(l == r)return;
        int mid = (l + r) >> 1;
        self(self,l,mid);
        self(self,mid + 1,r);
        sort(arr.begin() + l,arr.begin() + 1 + mid,cmpb);
        sort(arr.begin() + 1 + mid,arr.begin() + 1 + r,cmpb);
        int i = l;
        for(int j = mid + 1;j <= r;j++)
        {
            while(i <= mid && arr[i].b <= arr[j].b)
            {
                fi.add(arr[i].c,arr[i].cnt);
                i++;
            }
            arr[j].res += fi.query(1,arr[j].c);
        }
        for(int j = l;j < i;j++)fi.add(arr[j].c,-arr[j].cnt);
    };
    cdq(cdq,1,len);
    vector<int>ans(n);
    for(int i = 1;i <= len;i++)ans[arr[i].res + arr[i].cnt - 1] += arr[i].cnt;
    for(int i = 0;i < n;i++)
        cout << ans[i] << endl;

}
```